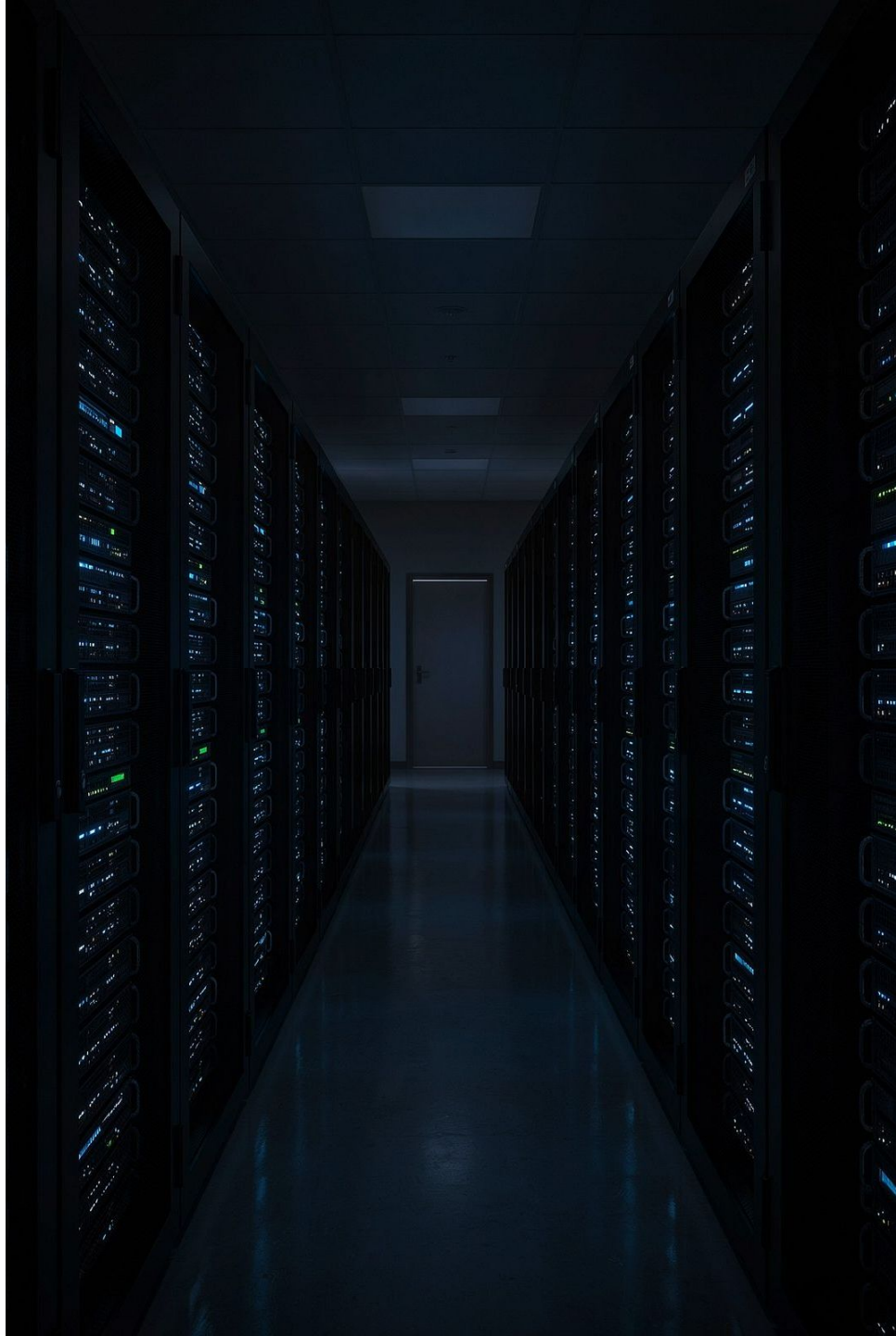
# Module 4: Automation & Incident Response with AIOps and GitHub

How modern engineering teams stop chasing fires — and start preventing them. This module walks through intelligent event management, automated workflows, and the tools that make incident response faster, smarter, and less painful.

AIOPS · GITHUB ACTIONS · INCIDENT RESPONSE



Module Roadmap

# What We'll Cover Today

Five interconnected topics that build on each other — from detecting a problem to automatically fixing it, all while keeping your team informed every step of the way.

01

## Intelligent Event & Incident Management

Root Cause Analysis, correlation, and smart alerting

02

## Automating Response Workflows

GitHub Actions as the engine of automation

03

## Tool Integrations

Jira, ServiceNow, and Slack working in concert

04

## Automated Testing & Validation

Safe deployments with automated guardrails

05

## Monitoring, Logging & Feedback

Continuous visibility and learning loops



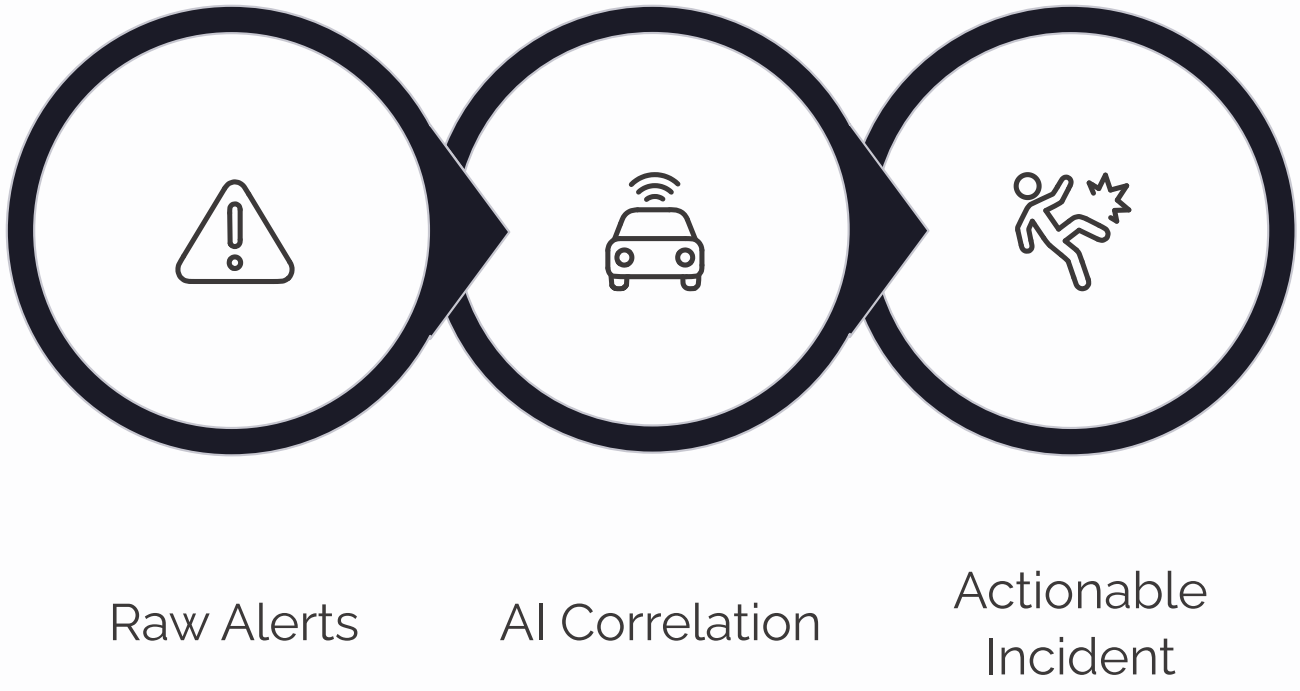
# The Old Way: Fighting Fires Manually

Imagine it's 2 AM. Your e-commerce site is down on Black Friday. Your team gets 500 alerts at once — CPU spikes, database timeouts, API errors. Nobody knows where to start. Engineers scramble, Slack is chaos, and customers are leaving. **This is life without AIOps.** The goal of this module is to show you how to move from reactive firefighting to proactive, automated incident response — so your team sleeps better and your users never notice the blip.

# Chapter 1: Intelligent Event & Incident Management

PART 1 OF 5

Before you can automate a response, you need to **understand what's actually happening**. AIOps transforms a flood of raw alerts into actionable intelligence.





# The Alert Flood Problem

A typical enterprise application generates **thousands of alerts per day**. Most of them are noise — or duplicates of the same underlying problem. When a database goes down, it triggers alerts from the database itself, every microservice that depends on it, the load balancer, the API gateway, and monitoring dashboards. That's one problem creating 50 alerts. AIOps uses machine learning to **correlate these signals**, group them into a single incident, and point engineers directly at the real culprit — the database — instead of the symptoms.

# Smarter Alerting: Less Noise, More Signal

Good alerting is about sending the **right alert to the right person at the right time** — not blasting everyone with every warning.



## Noise Suppression

Suppress redundant alerts from cascading failures. One incident ticket, not fifty Slack pings.



## Intelligent Routing

Route database alerts to the DB team, not the frontend engineers. Context-aware escalation paths.



## Severity Prioritization

AI ranks incidents by business impact — a payment gateway failure ranks higher than a dev environment warning.



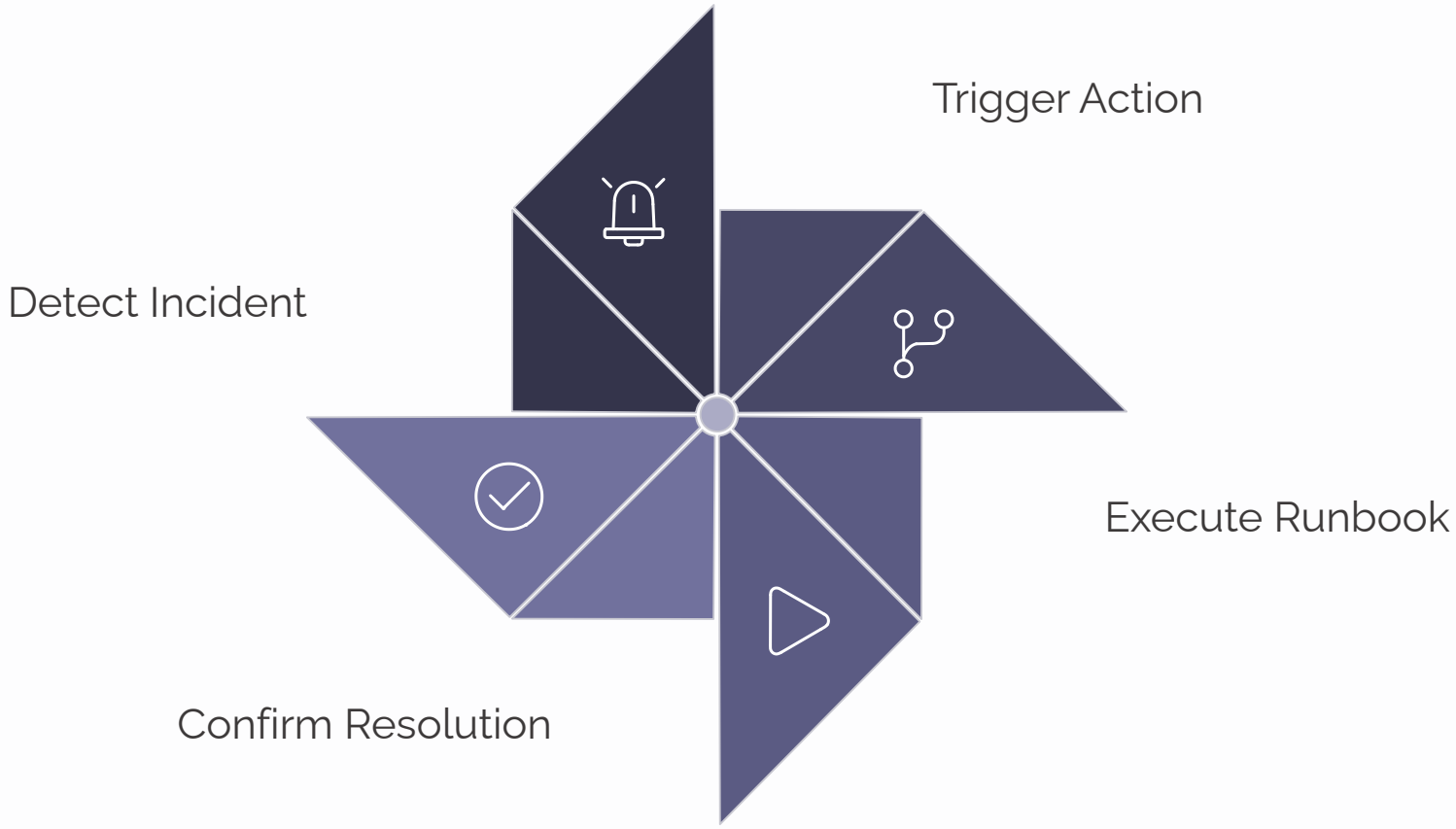
## On-Call Awareness

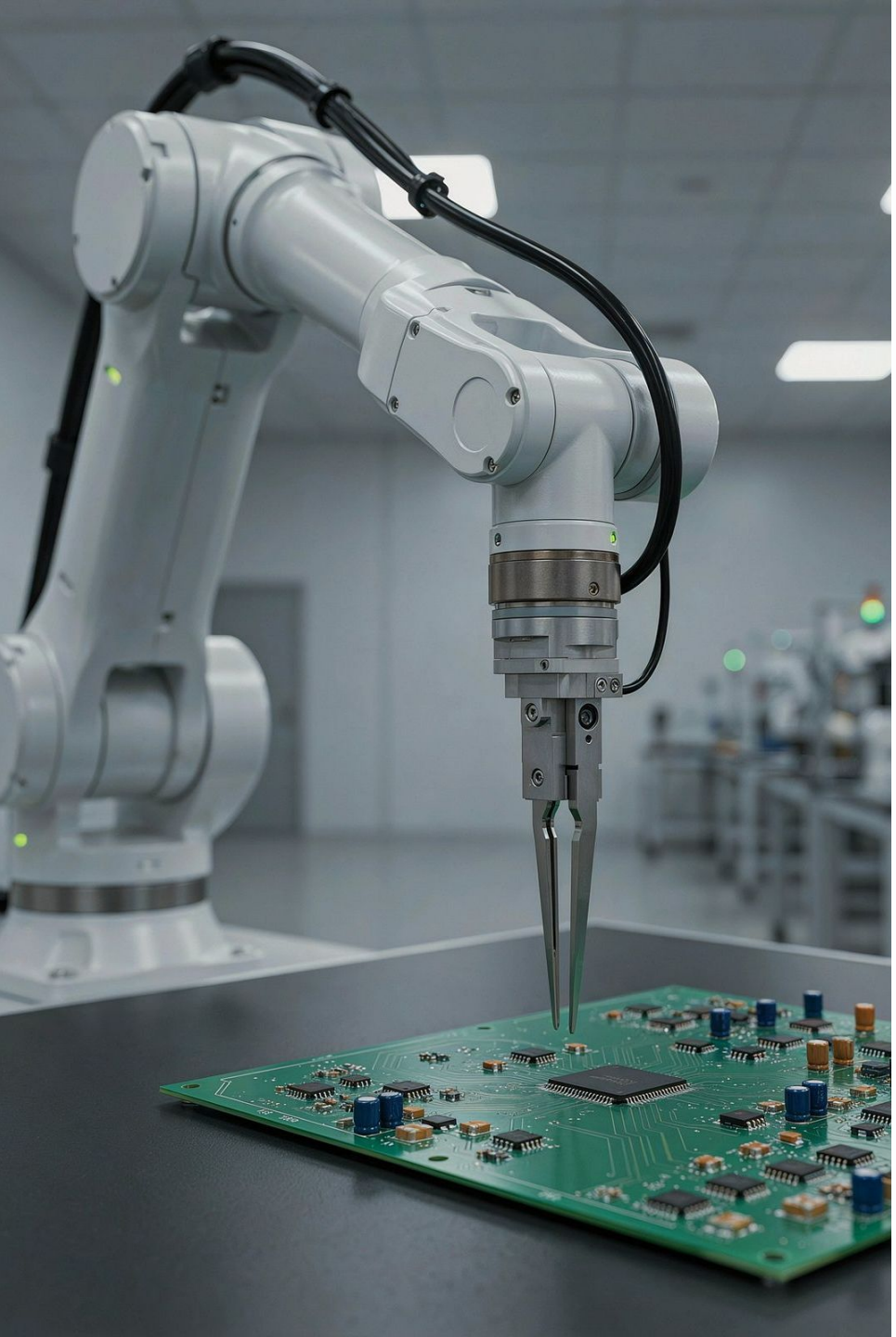
Integrates with PagerDuty schedules to page the engineer who is actually on duty — not their teammate on vacation.

# Chapter 2: Automating Response Workflows with GitHub Actions

PART 2 OF 5

Once an incident is detected and understood, the next question is: **can the system fix itself?** GitHub Actions is the automation engine that makes this possible — turning human runbooks into code.





# What Are GitHub Actions?

GitHub Actions is a CI/CD and automation platform built directly into GitHub. Think of it as a **programmable robot teammate** that wakes up whenever something happens — a code push, a pull request, a webhook from your monitoring tool — and executes a predefined set of tasks. These tasks live in YAML files called **workflows**, stored in your repository under `.github/workflows/`. This means your automation logic is version-controlled, auditable, and reviewed just like your application code. No more "who changed the Jenkins job?" mysteries.

# Automating Incident Response: A Real Scenario

Here's how GitHub Actions turns a 45-minute manual incident into a 90-second automated recovery:

1

### 🚨 T+0s: Alert Fires

Datadog detects memory usage at 95% on prod server. Webhook triggers GitHub Actions workflow.

2

### ⚙️ T+15s: Diagnosis Step

Workflow queries CloudWatch logs, identifies a memory-leaking process introduced in the last deployment.

3

### 🔄 T+45s: Auto-Remediation

Workflow triggers a rolling restart of the affected pods in Kubernetes. No human involved yet.

4

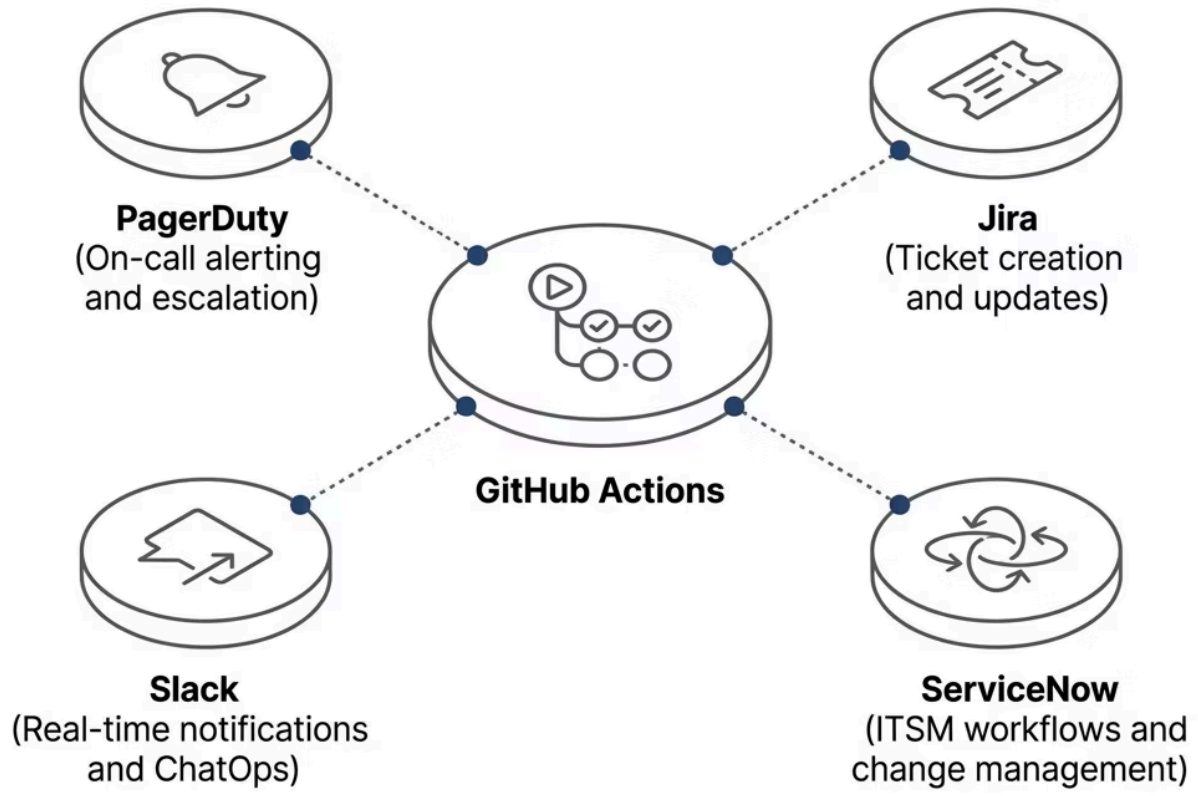
### ✅ T+90s: Confirm & Notify

Memory drops to 45%. Workflow posts resolution summary to Slack and closes the incident ticket in Jira.

# Chapter 3: Integrating GitHub Actions with Incident Tools

PART 3 OF 5

No tool works in isolation. The real power comes from **connecting GitHub Actions to the platforms your teams already live in** — so incidents are tracked, communicated, and resolved without switching context.



# Jira & ServiceNow: Keeping the Ticket Trail Alive

## Jira Integration

When an alert fires, a GitHub Actions workflow automatically creates a Jira issue with severity, affected service, and a link to the triggering alert. As the workflow progresses, it adds comments — "Rolling restart initiated at 14:32." When resolved, it transitions the ticket to Done. Your project board stays current with zero manual updates.

## ServiceNow Integration

For enterprise environments, ServiceNow governs change management. GitHub Actions can open a Change Request before a deployment, wait for ITSM approval, and only proceed once a human approves it in ServiceNow. This keeps automation compliant with governance policies — speed without sacrificing control.



# Slack & ChatOps: Your Incident War Room in Real-Time

Slack becomes the **live incident dashboard** for the whole team. GitHub Actions posts structured messages at every stage: incident opened, diagnosis running, remediation in progress, resolution confirmed. Engineers can even trigger actions *from Slack* — typing `/deploy rollback production` in a channel kicks off a GitHub Actions workflow without anyone touching a terminal. This ChatOps model keeps the entire team synchronized and creates a timestamped audit trail of every action taken during an incident — invaluable for post-mortems.

# Chapter 4: Automated Testing & Validation

PART 4 OF 5

Most incidents are caused by deployments. **Automated testing is your first line of defense** — catching problems before they ever reach production.

1

## Unit Tests

Does each function do what it's supposed to?

2

## Integration Tests

Do services talk to each other correctly?

3

## Infrastructure Validation

Is the Terraform plan safe to apply?

4

## Smoke Tests

Is the deployed app actually responding?

# Testing Infrastructure as Code: The Terraform Example

## The Problem Without Automation

A developer changes a security group rule in Terraform, merges it without review, and accidentally opens port 22 to the public internet. The change deploys. Now you have a compliance violation and a potential breach — discovered three days later in an audit.

## With GitHub Actions + Automated Validation

On every pull request, a workflow runs terraform plan and pipes the output through **Checkov** (a policy scanner). Checkov flags the open port 22 rule as a critical violation. The PR is blocked. The developer is notified with the exact line to fix. The change never reaches production. Problem solved before it existed.



# Canary Deployments & Automated Rollbacks

A **canary deployment** rolls out a new version to 5% of users first, while 95% still hit the stable version. GitHub Actions monitors key metrics — error rate, latency, CPU — for 10 minutes. If everything looks healthy, it automatically promotes the canary to 100%. If error rates spike above a threshold, it **automatically rolls back** to the previous version and files a Jira ticket with diagnostic data attached. Users on the 95% path experienced nothing. The 5% who hit the bad version got errors for under 10 minutes. This is automated validation working at its best.

# Chapter 5: Monitoring, Logging & Feedback Mechanisms

PART 5 OF 5

Automation without observability is flying blind. You need to **see what your systems are doing**, learn from every incident, and continuously improve your automated responses.

### Observe

Collect metrics, logs, and traces from all systems



### Detect

AI surfaces anomalies and triggers workflows

### Learn

Post-mortems and feedback improve future playbooks

### Respond

Automated playbooks execute remediation steps

# The Observability Stack: What to Measure

### 📊 Metrics

Numerical time-series data: CPU %, request latency (p99), error rate, throughput. Tools: Prometheus, Datadog, CloudWatch. These are your early warning signals.

### 📋 Logs

Timestamped text records of everything that happened. Tools: ELK Stack, Splunk, Loki. Logs answer "what exactly happened and when?" during a post-mortem.

### 🔍 Traces

End-to-end request journeys across microservices. Tools: Jaeger, Zipkin, Honeycomb. Traces answer "why is this API call slow?" across 10 different services.

Together, these three pillars — metrics, logs, and traces — give your AIOps platform the raw material it needs to detect, diagnose, and learn from every incident.

# Feedback Loops: Getting Smarter After Every Incident

Every resolved incident is a learning opportunity. GitHub Actions can automatically generate a **post-mortem draft** — pulling the incident timeline, the alerts that fired, the actions taken, and the resolution time — and posting it to Confluence or a GitHub issue. Teams review it, add human context, and then update their automation playbooks to handle the same scenario faster next time. Over months, this feedback loop transforms your runbooks from static documents into **living, battle-tested automation**. The system literally gets smarter with every outage it survives.



# Key Takeaways: From Chaos to Control

Here's what modern AIOps and GitHub Actions give your team — in plain terms:

### Fewer 2 AM pages

AI correlation reduces alert noise by up to 90%, and automated remediation resolves common incidents before a human is ever paged.

### Faster recovery

Automated workflows execute in seconds what used to take engineers 30–60 minutes of manual diagnosis and action.

### Safer deployments

Automated testing, canary rollouts, and instant rollbacks mean shipping code is no longer a white-knuckle event.

### A system that learns

Feedback loops and post-mortem automation mean your incident response gets measurably better every single month.